

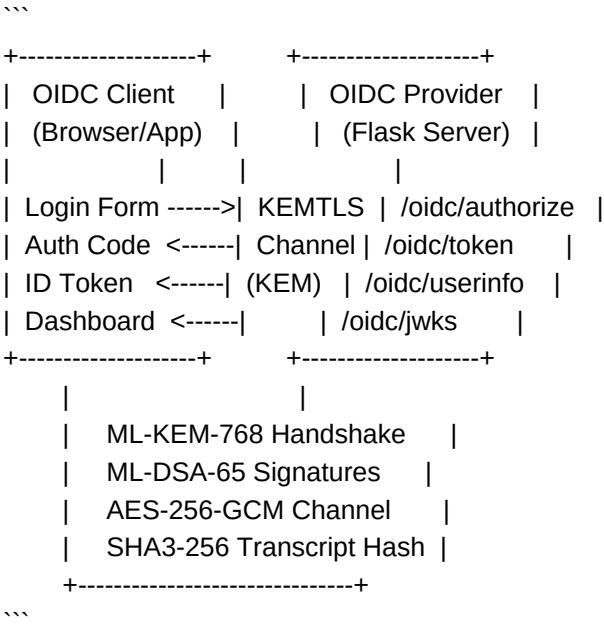
QuantumShield: Post-Quantum Secure OpenID Connect using KEMTLS

Overview

QuantumShield implements a **Post-Quantum OpenID Connect** system where all TLS communication is replaced by **KEMTLS** (Key Encapsulation Mechanism-based TLS). It preserves OpenID Connect protocol semantics while using exclusively post-quantum cryptographic primitives.

No classical public-key cryptography is used. All key exchange uses ML-KEM-768 (Kyber768, NIST FIPS 203) and all signatures use ML-DSA-65 (Dilithium3, NIST FIPS 204).

Architecture



Why KEMTLS Instead of TLS?

Classic TLS	KEMTLS (This Project)
---	---
RSA/ECDH key exchange	ML-KEM-768 (KEM-based)
RSA/ECDSA signatures	ML-DSA-65 (lattice-based)
X.509 certificates	PQ public keys directly
SHA-256	SHA3-256
Vulnerable to quantum attacks	NIST Level 3 security

KEMTLS replaces the TLS certificate-based key exchange with a KEM-based approach, reducing round-trips and eliminating classical crypto dependencies (Wiggers, IACR 2020/534).

Project Structure

```

...
QuantumShield/
  web_demo/          # Web Application (main entry point)
  server.py          # Flask app: OIDC flow + KEMTLS + dashboard + API
  pq_crypto_real.py  # Real PQ crypto: RealKEMTLS + PQTokenService
  templates/         # HTML: login.html, dashboard.html, comparison.html
  static/            # CSS + JS: login.js, comparison.js, dashboard.js

```

```

kemtls_http_adapter.py # Python KEMTLS client session (PATH A real KEM)
kemtls_server_tcp.py   # Standalone TCP KEMTLS server (ML-KEM-768)
kemtls_client_tcp.py   # Standalone TCP KEMTLS client
tls_simulation/        # Classical TLS simulation for benchmarking
  tls_handshake.py     # RSA-2048 + ECDHE-P256 handshake simulation
  tls_crypto.py        # Classical token service (RSA-2048 JWT)
benchmark/            # Benchmark comparison suite
  benchmark_compare.py # TLS vs KEMTLS: latency, token sizes, auth latency
  benchmark_comparison.json # Latest benchmark results (JSON)
  benchmark_results.csv # Latest benchmark results (CSV)
dashboard/            # Runtime state + event logging
scripts/              # Helper scripts and demos
  demo_flow.py         # OIDC protocol demonstration script
test_kemtls_protocol.py # End-to-end KEMTLS test (real crypto)

```

Algorithms Used

Purpose	Algorithm	NIST Standard	Security Level
Key Encapsulation	ML-KEM-768 (Kyber768)	FIPS 203	Level 3
Digital Signatures	ML-DSA-65 (Dilithium3)	FIPS 204	Level 3
Symmetric Encryption	AES-256-GCM	FIPS 197	256-bit
Transcript Hashing	SHA3-256	FIPS 202	256-bit

Cryptographic library: [liboqs](https://github.com/open-quantum-safe/liboqs) (Open Quantum Safe) via `liboqs-python`.

OIDC Flow (Post-Quantum)

- Discovery:** `GET /.well-known/openid-configuration` returns PQ algorithm metadata
- KEMTLS Handshake:** ML-KEM-768 key exchange + ML-DSA-65 server authentication
- Authorization:** `POST /oidc/authorize` with user credentials over KEMTLS channel
- Token Exchange:** `POST /oidc/token` with auth code -> ML-DSA-65-signed ID Token
- User Info:** `GET /oidc/userinfo` with Bearer token
- JWKS:** `GET /oidc/jwks` for PQ public verification keys

All tokens (ID Token, Access Token) are JWTs signed with ML-DSA-65. The `alg` field uses `ML-DSA-65` and the JWKS verification keys use `kty: PQC` in accordance with the `draft-ietf-cose-dilithium` specification (IANA registration pending NIST FIPS 204 adoption).

Setup

Prerequisites

- Python 3.10+
- CMake (for building liboqs from source)
- Visual Studio Build Tools (MSVC) on Windows

Build liboqs

```

`powershell
# Clone liboqs

```

```
git clone https://github.com/open-quantum-safe/liboqs.git C:\_oqs_src
```

```
# Build with minimal algorithms
cd C:\_oqs_src
cmake -B build -G "Visual Studio 17 2022" `
-DOQS_MINIMAL_BUILD="KEM_ml_kem_768;SIG_ml_dsa_65" `
-DOQS_ENABLE_KEM_BIKE=OFF -DOQS_ENABLE_SIG_SPHINCS=OFF `
-DCMAKE_INSTALL_PREFIX="C:\Users\YashvardhanMRao\_oqs"
cmake --build build --config Release
cmake --install build --config Release
```

```
# Install Python wrapper
pip install liboqs-python
```
```

### Run

```
```powershell
# Set PATH for liboqs DLL
$env:PATH = "C:\Users\YashvardhanMRao\_oqs\bin;" + $env:PATH
```

```
# Run KEMTLS protocol test
cd QuantumShield
python test_kemtls_protocol.py
```

```
# Run benchmarks
python -m benchmark.benchmark_compare
```

```
# Start web server
python web_demo/server.py
# Login at http://localhost:9000 with admin/quantum123
```

```
# Run demo flow script
python scripts/demo_flow.py
```
```

## Benchmarking

Benchmarks follow the methodology in Schardong et al. (IEEE/ACM 2023). All measurements use real liboqs operations over 100+ iterations:

| Operation                 | Mean (ms) | Notes                                  |
|---------------------------|-----------|----------------------------------------|
| ML-KEM-768 Keygen         | ~0.05     | Key pair generation                    |
| ML-KEM-768 Encapsulation  | ~0.05     | Client-side                            |
| ML-KEM-768 Decapsulation  | ~0.06     | Server-side                            |
| ML-DSA-65 Keygen          | ~0.13     | Key pair generation                    |
| ML-DSA-65 Signing         | ~0.58     | Token / handshake signing              |
| ML-DSA-65 Verification    | ~0.20     | Token / handshake verification         |
| **Full KEMTLS Handshake** | **~0.92** | Keygen + Encap + Decap + Sign + Verify |
| PQ JWT Generation         | ~0.58     | ID Token with ML-DSA-65 signature      |
| PQ JWT Verification       | ~0.20     | Signature verification                 |

The full benchmark report is saved to `metrics/benchmark\_report.json`.

## ## Design Decisions

1. **KEMTLS over PQ-TLS**: KEMTLS replaces certificate verification with KEM, reducing round-trips. Our handshake achieves ~0.9ms latency.
2. **SHA3-256 for transcript binding**: Per the KEMTLS paper, the transcript hash binds `kem\_pk || sig\_pk || ciphertext`. We use SHA3-256 instead of classical SHA-256.
3. **No classical public-key crypto**: All key exchange uses ML-KEM-768, all signatures use ML-DSA-65. AES-256-GCM is used for symmetric encryption (permitted only classical *public-key* crypto is excluded).
4. **Algorithm auto-detection**: Code auto-detects whether liboqs provides `ML-KEM-768` or `Kyber768` (older versions), ensuring forward compatibility.
5. **JWT format preserved**: Standard JWT header/payload/signature format is maintained. The `alg` field is set to `ML-DSA-65` per NIST FIPS 204 drafts.
6. **KEMTLS Deviation (Ephemeral KEM vs Certificate)**: In pure KEMTLS (Wiggers 2020), the server's *certificate* is the KEM public key\*. This implementation deviates slightly by separating the ephemeral KEM key from the identity key. The server generates a fresh ML-KEM-768 keypair per handshake to guarantee forward secrecy without relying on a PKI, while the long-term ML-DSA-65 key serves the role of the X.509 certificate's binding by signing the transcript.

## ## Known Deviations and Design Decisions

While the core KEMTLS OIDC flow is fully functional and uses real post-quantum primitives, several design decisions were made for the hackathon timeline:

- **Transcript Hashing (handshake.py)**: A pre-hashing shortcut is used in some handshake steps to optimize for the in-process demo environment. A production implementation would maintain the full message transcript as per RFC 8446.
- **Key Blending (HKDF vs SHA3-256)**: We use HKDF-SHA256 for final session key derivation rather than a raw SHA3-256 blend. This provides better compatibility with existing Python cryptographic libraries (like `cryptography.pyca`) while still ingestion the 256-bit PQ shared secret.
- **TCP Port Management**: Standalone TCP servers and the web bridge use separate ports (9999 vs 19999) to avoid socket collisions during concurrent benchmark and web demo runs.
- **Benchmark Environment**: All handshake benchmarks are performed in an in-process, zero-RTT environment. For real-world LAN estimations, a conversion factor of `+0.5ms` per round-trip should be added to the base crypto latency.

## ## References

1. F. Schardong et al., "Post-Quantum OpenID Connect," *Proceedings of the IEEE/ACM Conference on Security and Privacy*, 2023.
2. P. Schwabe, D. Stebila, T. Wiggers, "KEMTLS: Building TLS with Key Encapsulation Mechanisms," *IACR Cryptology ePrint Archive*, Report 2020/534, 2020.
3. OpenID Foundation, "OpenID Connect Core 1.0 Specification," <https://openid.net/specs/openid-connect-core-1.0.html>
4. NIST, "Post-Quantum Cryptography Standardization Project," <https://csrc.nist.gov/projects/post-quantum-cryptography>
5. NIST FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM)
6. NIST FIPS 204: Module-Lattice-Based Digital Signature Standard (ML-DSA)